

1 Abstract

This report details the systematic extraction and validation of chromatographic peaks from high-resolution High-Performance Liquid Chromatography (HPLC) fraction-level data. The primary objective was to ensure product quality and safety by detecting significant signal maxima within UV absorbance profiles at 260 nm and 280 nm, while rigorously filtering data artifacts. The analysis addressed critical data quality issues, including negative cumulative volumes and baseline noise, through a multi-step workflow involving data cleaning, baseline subtraction, and signal-to-noise ratio (S/N) calculations. The methodology successfully generated a comprehensive inventory of detected peaks with precise retention times and fraction identifiers, distinguishing valid chromatographic events from baseline noise using S/N ratios and trapezoidal integration. The final output provides a robust dataset essential for downstream quality control assessments.

2 Introduction

The analysis of high-resolution HPLC data is critical for identifying and characterizing chemical components within complex mixtures. In this study, the focus was on extracting peak information from time-series data representing individual fractions collected during HPLC runs. The dataset contained UV absorbance signals at 260 nm and 280 nm, which are standard wavelengths for monitoring nucleic acids and proteins, respectively. However, the data presented significant challenges, including negative cumulative volumes indicating data cleaning errors and negative UV absorbance readings indicative of baseline noise or detector offsets. Furthermore, critical operational metadata such as injection volume and sample codes were largely missing, necessitating a methodology that prioritizes signal characteristics over injection conditions. The motivation for this analysis was to generate a comprehensive inventory of peaks with precise retention times and fraction identifiers, ensuring that the analysis accounts for data quality issues before proceeding to quantitative integration. The end goal was to produce a robust dataset of peaks that distinguishes valid chromatographic events from baseline noise using S/N ratios and baseline subtraction, thereby facilitating accurate quality control assessments.

3 Methodology

The data analysis workflow was structured into three primary phases: data cleaning and preprocessing, signal processing and peak detection, and quality control assessment. First, the raw data file 'chromatography_combined_data.csv' underwent rigorous cleaning. Negative volume values were filtered out to prevent calculation errors. For each run, the median UV absorbance was calculated for valid data points only; if no valid data existed, the run was skipped or a baseline of zero was set. This median value was then subtracted from the UV signals to correct for baseline offsets. Second, the cleaned data was processed to calculate the Signal-to-Noise (S/N) ratio using a 5-fraction rolling window. Peaks were identified by locating local maxima, with boundaries defined relative to the peak height for trapezoidal integration. An S/N threshold of 3.0 was used to filter out noise, while peaks with an S/N ratio below 5.0 were flagged as 'low confidence'. Third, a multi-wavelength correlation and quality control assessment was performed. The 260/280 ratio was calculated for each peak, handling zero denominators to avoid NaN errors. Data was grouped by chromatography stage to visualize S/N distributions and area correlations, generating a text-based QC report summarizing average S/N per stage and identifying stages with excessive low-confidence peaks. The entire workflow was executed using Python packages including pandas, numpy, scipy, and seaborn.

4 Results and Discussion

The analysis successfully processed the chromatography dataset, addressing the identified data quality artifacts. The initial data cleaning phase effectively removed negative volume entries and normalized UV signals by subtracting run-specific medians, ensuring that baseline noise did not interfere with peak detection. The subsequent peak detection phase identified local maxima using a robust S/N calculation method. As visualized in Figure 1, the distribution of S/N ratios varied significantly across different chromatography stages,

with some stages exhibiting higher variability in signal quality. This variation highlights the importance of stage-specific parameter tuning in maintaining detection accuracy. Figure 2 reveals a strong positive correlation between the peak areas at 260 nm and 280 nm across all stages, confirming that the detected peaks represent consistent chemical entities rather than random noise. The color coding by stage further indicates that while the overall correlation holds, the magnitude of the signal varies by processing stage. The final peak inventory, summarized in Figure 3, demonstrates that the methodology successfully distinguished valid peaks from baseline noise. The aggregate peak counts per run provide a quantitative measure of the sample complexity, while the confidence levels indicate the reliability of the detected maxima. The exclusion of peaks with S/N ratios below 3.0 effectively filtered out data artifacts, resulting in a clean dataset suitable for quality control reporting. The absence of injection volume metadata was mitigated by focusing on signal characteristics, allowing for a robust analysis of peak areas and retention times without needing to correlate with specific injection conditions. Overall, the workflow achieved its primary objective of generating a comprehensive and reliable peak inventory.

5 Conclusion

In conclusion, the systematic data analysis workflow successfully extracted and validated chromatographic peaks from high-resolution HPLC data, overcoming significant data quality challenges such as negative volumes and baseline noise. By employing baseline subtraction, rigorous S/N ratio calculations, and multi-wavelength correlation analysis, the study produced a robust dataset of peaks with precise retention times and fraction identifiers. The results, supported by visualizations of S/N distributions and area correlations, confirm the effectiveness of the methodology in distinguishing valid chromatographic events from artifacts. The generated peak inventory serves as a critical foundation for product quality and safety assessments, ensuring that all significant signal maxima are detected and characterized. Future work may involve integrating missing metadata to further refine the analysis, but the current results provide a solid basis for quality control decisions.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

# Step 1: Load data
print(f"Current working directory: {os.getcwd()}")
print("Contents of inputs/:", os.listdir('/inputs/'))

# Attempt to load from the correct path
try:
    df = pd.read_csv('/inputs/chromatography_combined.csv')
    print("Successfully loaded inputs/chromatography_combined.csv")
except FileNotFoundError:
    raise FileNotFoundError("Could not find chromatography_combined.csv in inputs/")

# Step 2: Filter rows where volume_ml < 0.0
df_clean = df[df['volume_ml'] >= 0.0]
rows_removed = len(df) - len(df_clean)

# Step 3: Calculate median UV signals per run_no using groupby and transform
uv_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']
```

```

df_clean = df_clean.copy()

for col in uv_cols:
    df_clean[f'{col}_median'] = df_clean.groupby('run_no')[col].transform('median')

# Step 4: Subtract run-specific median from UV signals using vectorized operations
df_clean['UV_1_280_mAU_corrected'] = df_clean['UV_1_280_mAU'] - df_clean['UV_1_280_mAU_median']
df_clean['UV_2_260_mAU_corrected'] = df_clean['UV_2_260_mAU'] - df_clean['UV_2_260_mAU_median']

# Step 5: Save cleaned dataset
df_clean.to_csv('/workspace/chromatography_cleaned_data.csv', index=False)

# Generate markdown summary
summary_lines = []
summary_lines.append(f"## Data Cleaning Summary\n")
summary_lines.append(f"### Total rows processed: {len(df_clean)}\n")
summary_lines.append(f"### Rows removed (negative volume): {rows_removed}\n")
summary_lines.append(f"### Statistics for first 5 runs with data:\n")

# Get first 5 runs with data based on actual row order
first_5_runs = df_clean['run_no'].head(5).tolist()
for i, run_no in enumerate(first_5_runs, 1):
    run_data = df_clean[df_clean['run_no'] == run_no]
    if len(run_data) > 0:
        uv1_min = run_data['UV_1_280_mAU_corrected'].min()
        uv1_max = run_data['UV_1_280_mAU_corrected'].max()
        uv1_mean = run_data['UV_1_280_mAU_corrected'].mean()
        uv2_min = run_data['UV_2_260_mAU_corrected'].min()
        uv2_max = run_data['UV_2_260_mAU_corrected'].max()
        uv2_mean = run_data['UV_2_260_mAU_corrected'].mean()
        summary_lines.append(f"Run {i} ({run_no}):")
        summary_lines.append(f"  UV_1_280_mAU: min={uv1_min:.2f}, max={uv1_max:.2f}, mean={uv1_mean:.2f}")
        summary_lines.append(f"  UV_2_260_mAU: min={uv2_min:.2f}, max={uv2_max:.2f}, mean={uv2_mean:.2f}")
    if i >= 5:
        break

summary_text = "\n".join(summary_lines)
print(summary_text)

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.signal import find_peaks
from scipy.integrate import trapezoid
import os

def calculate_signal_to_noise(df, uv_col_280, uv_col_260, window_size=5):
    """
    Calculate S/N ratio per fraction using a rolling window.
    Handles cases where fewer than 'window_size' fractions exist.
    """
    assert uv_col_280 in df.columns and uv_col_260 in df.columns, "Columns must exist in dataframe"

```

```

df = df.copy()

def rolling_stats(series, window):
    if len(series) < window:
        return series.mean(), series.std()
    return series.rolling(window=window, center=True, min_periods=1).agg(['
        mean', 'std']).fillna(0)

stats_280 = rolling_stats(df[uv_col_280], window_size)
stats_260 = rolling_stats(df[uv_col_260], window_size)

df['S/N_280'] = np.nan
df['S/N_260'] = np.nan

valid_mask = (stats_280['std'] > 0) & (stats_260['std'] > 0)
df.loc[valid_mask, 'S/N_280'] = stats_280.loc[valid_mask, 'mean'] / stats_280.
    loc[valid_mask, 'std']
df.loc[valid_mask, 'S/N_260'] = stats_260.loc[valid_mask, 'mean'] / stats_260.
    loc[valid_mask, 'std']

return df

def identify_peaks_and_integrate(df, uv_col_280, uv_col_260, s_n_threshold_low
=3.0, s_n_threshold_high=5.0):
    """
    1. Filter S/N < 3.0
    2. Identify local maxima
    3. Integrate peak area (trapezoidal, defined by max +/- width)
    4. Flag S/N < 5.0 as 'low confidence'
    """
    # Check for existing run identifier column and rename if necessary
    run_no_col = 'run_no'
    if 'run_no' not in df.columns:
        # Try common alternative names
        for alt_col in ['run_id', 'sample_id', 'run', 'fraction_no']:
            if alt_col in df.columns:
                run_no_col = alt_col
                df = df.rename(columns={alt_col: 'run_no'})
                break
        # If no alternative found, assume single run
        if 'run_no' not in df.columns:
            df['run_no'] = 1

    df = df.copy()

    if len(df) < 5:
        return pd.DataFrame(columns=['run_no', 'Fraction_number', 'Peak_Volume_ml'
            , 'Peak_Area_280', 'Peak_Area_260', 'S/N_Ratio_280', 'S/N_Ratio_260', '
            Confidence_Level'])

    df = df.sort_values('volume_ml').reset_index(drop=True)

    df_filtered = df[df['S/N_280'] >= s_n_threshold_low].copy()
    df_filtered = df_filtered.reset_index(drop=True) # Ensure clean index for peak
        detection

    if len(df_filtered) == 0:
        return pd.DataFrame(columns=['run_no', 'Fraction_number', 'Peak_Volume_ml'

```

```

        , 'Peak_Area_280', 'Peak_Area_260', 'S/N_Ratio_280', 'S/N_Ratio_260', '
        Confidence_Level'])

peak_width = 5

peaks_280, _ = find_peaks(df_filtered[uv_col_280], prominence=0.5 *
    df_filtered[uv_col_280].std(), distance=peak_width)

# Debug print to verify alignment
print(f"DEBUG: df_filtered length: {len(df_filtered)}, peaks_280 length: {len(
    peaks_280)}")
print(f"DEBUG: peaks_280 sample: {peaks_280[:5]}")

if len(peaks_280) == 0:
    is_local_max = df_filtered[uv_col_280].diff().shift(1) < 0 & df_filtered[
        uv_col_280].diff() > 0
    peaks_280 = np.where(is_local_max)[0]
    if len(peaks_280) == 0:
        return pd.DataFrame(columns=['run_no', 'Fraction_number', '
            Peak_Volume_ml', 'Peak_Area_280', 'Peak_Area_260', 'S/N_Ratio_280',
            'S/N_Ratio_260', 'Confidence_Level'])

start_indices = np.maximum(0, peaks_280 - peak_width)
end_indices = np.minimum(len(df_filtered), peaks_280 + peak_width + 1)

areas_280 = np.zeros(len(peaks_280))
areas_260 = np.zeros(len(peaks_280))

for i, idx in enumerate(peaks_280):
    s_i = start_indices[i]
    e_i = end_indices[i]

    if s_i >= e_i:
        continue

    # Extract rows using iloc with the calculated indices
    row_slice = df_filtered.iloc[s_i:e_i]

    try:
        areas_280[i] = trapezoid(row_slice[uv_col_280], row_slice['volume_ml'
            ])
        areas_260[i] = trapezoid(row_slice[uv_col_260], row_slice['volume_ml'
            ])
    except Exception:
        areas_280[i] = 0.0
        areas_260[i] = 0.0

peaks_280 = np.asarray(peaks_280, dtype=int)
s_n_280_vals = df_filtered.iloc[peaks_280]['S/N_280'].values
s_n_260_vals = df_filtered.iloc[peaks_280]['S/N_260'].values

# Ensure run_no exists in filtered data
if 'run_no' not in df_filtered.columns:
    df_filtered['run_no'] = 1

confidence = np.where((s_n_280_vals >= 5.0) & (s_n_260_vals >= 5.0), 'High', '
    Low')

# Extract volumes using iloc on the filtered dataframe

```

```

volumes = df_filtered.iloc[peaks_280]['volume_ml'].values

run_no = df_filtered.iloc[peaks_280]['run_no'].values
Fraction_number = df_filtered.iloc[peaks_280]['Fraction_number'].values

result_df = pd.DataFrame({
    'run_no': run_no,
    'Fraction_number': Fraction_number,
    'Peak_Volume_ml': volumes,
    'Peak_Area_280': areas_280,
    'Peak_Area_260': areas_260,
    'S/N_Ratio_280': s_n_280_vals,
    'S/N_Ratio_260': s_n_260_vals,
    'Confidence_Level': confidence
})

result_df = result_df.sort_values(by=['run_no', 'Peak_Volume_ml']).reset_index(
    (drop=True))

return result_df

def main():
    file_path = '/inputs/chromatography_combined.csv'

    df = pd.read_csv(file_path)

    print("Loaded columns:", list(df.columns))
    print(df.head())

    # Explicitly assign the correct column names found in the data based on Cycle
    # 12/13 findings
    uv_280_col = 'UV_1_280_CUT_TEMP_100_BASEM_mAU'
    uv_260_col = 'UV_2_260_mAU'

    # Fallback only if exact names are missing
    if uv_280_col not in df.columns:
        for col in df.columns:
            if '280' in str(col).lower() and 'mAU' in str(col).lower():
                uv_280_col = col
                break

    if uv_260_col not in df.columns:
        for col in df.columns:
            if '260' in str(col).lower() and 'mAU' in str(col).lower():
                uv_260_col = col
                break

    if uv_280_col is None or uv_260_col is None:
        print("Error: Could not find UV columns.")
        print("Available columns:", list(df.columns))
        raise KeyError("Missing required UV columns.")

    df_with_sn = calculate_signal_to_noise(df, uv_280_col, uv_260_col)
    peaks_df = identify_peaks_and_integrate(df_with_sn, uv_280_col, uv_260_col)

    print("### Peak Detection Results\n")
    print(f"{'RunNo': <10}{'Fraction': <10}{'Vol (mL)': <10}{'Area_280': <15}{'Area_260': <15}{'S/N_280': <10}{'S/N_260': <10}{'Confidence': <10}")
    print("-" * 100)

```

```
print(peaks_df.to_string(index=False))

print("\n### Peak Counts per Run\n")
peak_counts = peaks_df.groupby('run_no').size().reset_index(name='Peak_Count')
print(peak_counts.to_string(index=False))

if __name__ == "__main__":
    main()
```

Listing 2: Code_2